

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №1

Выполнил:

Алексеев Андрей

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках практической работы требовалось выбрать предметную область, спроектировать модель данных и подготовить ER-диаграмму базы данных. В качестве предметной области был выбран сайт для поиска работы. Система должна поддерживать регистрацию и вход пользователей, личный кабинет соискателя с резюме, поиск вакансий с фильтрацией, детальную карточку вакансии, а также личный кабинет работодателя для управления вакансиями.

Ход работы

На первом этапе была выделена предметная область и определены основные роли системы: соискатель, работодатель и администратор. После этого были выделены основные сущности, которые требуются для реализации сценариев сайта поиска работы: пользователь, компания, профиль работодателя, резюме, вакансия, отклик, избранная вакансия, просмотр вакансии, справочник отраслей, справочник уровней опыта и отдельная таблица назначений HR по вакансии.

Далее была спроектирована ER-модель. В модели использован единый snake_case-нейминг, таблицы названы в единственном числе, а для всех сущностей введена пара идентификаторов: внутренний id BIGINT и внешний public_id UUID. Такое решение удобно для backend-реализации на Spring Boot Kotlin: связи между таблицами строятся на BIGINT, а во внешнем API используются только UUID.

В модели были сохранены аудиторские поля `created_at` и `updated_at`, а для таблицы `vacancy_assignment` также добавлено поле `version` для безопасного обновления назначений HR. При этом бизнес-проверки, такие как допустимые значения статусов, ролей, диапазонов зарплаты и форматов работы, намеренно оставлены на стороне backend-кода. База данных при таком подходе выполняет роль хранилища и механизма ссылочной целостности, но не содержит бизнес-логики через CHECK и DB ENUM.

Ключевая идея итоговой модели состоит в том, что вакансия одновременно имеет двух разных "владельцев" в разных смыслах:

- бизнес-владельцем вакансии является компания через `vacancy.company_id`;
- конкретные HR-сотрудники, ведущие вакансию, хранятся в таблице `vacancy_assignment`.

Дополнительно в самой таблице `vacancy` было оставлено поле `vacancy_assignment_id`, которое указывает на основное назначение HR по вакансии, то есть на главного ответственного сотрудника. Это позволяет не смешивать компанию как владельца вакансии и HR-стек как команду сопровождения.

С точки зрения связей в модели были зафиксированы следующие зависимости:

- один пользователь может иметь один `employer_profile` и несколько `resume`;

- одна компания может иметь несколько `employer_profile` и несколько `vacancy`;
- одна вакансия принадлежит одной компании, одной отрасли и одному уровню опыта;
- одна вакансия может иметь несколько назначений HR через `vacancy_assignment`;
- одно назначение связывает одну вакансию и один профиль работодателя;
- один отклик связывает одно резюме и одну вакансию;
- пользователь может добавлять вакансии в избранное и формировать историю просмотров.

При проектировании модели была также учтена будущая реализация поиска вакансий. В таблице `vacancy` предусмотрены поля, необходимые для фильтрации и сортировки: отрасль, уровень опыта, город, зарплатный диапазон, тип занятости, формат работы и статус. Для резюме предусмотрены аналогичные поля ожиданий соискателя. Для этих колонок заложены индексы, чтобы в дальнейшем `backend` мог эффективно выполнять типовые запросы поиска и выдачи.

Итоговая диаграмма была оформлена в `draw.io` в виде ER-модели. В ней таблицы расположены в читабельном виде, связи показаны в нотации "вороньих лапок", а допустимые значения для полей `VARCHAR`, играющих роль псевдо-enum, вынесены в отдельный блок.

В качестве итоговых материалов по работе были подготовлены ER-диаграмма базы данных и текстовое описание модели данных.

Обоснование выбора таблиц

Таблица `user` нужна как базовая учетная запись пользователя. В ней хранятся данные, общие для всех ролей: ФИО, email, телефон, пароль и роль. Именно от этой сущности начинается авторизация, аутентификация и связь со всеми пользовательскими действиями.

Таблица `company` вынесена отдельно, потому что компания и аккаунт работодателя не являются одной сущностью. Компания — это бизнес-объект, который владеет вакансиями, а пользователь-работодатель только действует от ее имени.

Таблица `employer_profile` связывает пользователя с компанией. Она показывает, что конкретный пользователь является работодателем, относится к определенной компании и имеет свою должность внутри компании. Это позволяет не смешивать данные аккаунта и данные бизнес-профиля.

Таблицы `industry` и `experience_level` были вынесены как отдельные справочники. Такое решение позволяет избежать хранения произвольных строк в вакансиях, обеспечивает нормализацию данных и упрощает построение фильтров поиска.

Таблица `resume` выделена отдельно, потому что у одного соискателя может быть несколько резюме под разные направления. В ней хранится содержательная часть профиля кандидата: желаемая должность, опыт, образование, навыки, зарплатные ожидания и параметры поиска работы.

Таблица `vacancy` является центральной сущностью модели. Она хранит карточку вакансии и все данные, которые нужны для публичного поиска и просмотра. Вакансия привязана именно к компании, потому что публикуется от имени компании.

Таблица `vacancy_assignment` была добавлена для хранения HR-стека вакансии. Она позволяет привязывать к одной вакансии нескольких сотрудников работодателя и назначать им роли, например `PRIMARY`, `RECRUITER` или `COORDINATOR`. Такое решение позволяет хранить не просто абстрактного "автора вакансии", а реальную команду сопровождения.

Таблица `application` нужна для хранения откликов. Она связывает резюме и вакансию, а пользователь отклика при этом однозначно определяется через `resume.user_id`. Это позволило не дублировать `user_id` в самой таблице откликов.

Таблица `favorite_vacancy` отвечает за состояние "избранное". Она связывает пользователя и вакансию и хранит факт того, что пользователь сохранил вакансию для себя.

Таблица `vacancy_view` отвечает за историю просмотров. Она вынесена отдельно, потому что просмотр — это событие, а не состояние. Один и тот же пользователь может смотреть одну и ту же вакансию несколько раз, и каждая запись должна сохраняться отдельно.

Таким образом, каждая таблица отвечает за отдельный фрагмент предметной области:

- user — учетная запись пользователя;
- company — карточка компании;
- employer_profile — профиль работодателя внутри компании;
- industry — справочник отраслей;
- experience_level — справочник уровней опыта;
- resume — резюме соискателя;
- vacancy — вакансия компании;
- vacancy_assignment — назначение HR на вакансию;
- application — отклик на вакансию;
- favorite_vacancy — избранная вакансия пользователя;
- vacancy_view — просмотр вакансии пользователем.

Такой набор таблиц покрывает все основные бизнес-сценарии сайта для поиска работы и при этом остается удобным для реализации через Spring Boot Kotlin, Hibernate/JPA и PostgreSQL.

Вывод

В результате работы была спроектирована целостная модель данных для сервиса поиска работы. Модель покрывает сценарии как соискателя, так и работодателя, разделяет владельца вакансии и HR-стек, не смешивает разные сущности между собой и хорошо подходит для дальнейшей backend-реализации на Spring Boot Kotlin и PostgreSQL. Подготовленная ER-диаграмма может использоваться как основа для реализации миграций базы данных, доменной модели и REST API.